

JWT Authentication in FastAPI

A complete learning document

Built during the Wandr traveller app — Week 3

Stack used

FastAPI • SQLAlchemy • SQLite • pwdlib (Argon2) • PyJWT • pydantic-settings

1. Authentication Basics

Authentication is the process of verifying that a user is who they claim to be. Before building any auth system, you need to understand three fundamental concepts: what gets stored, what gets sent, and what gets checked.

1.1 The core problem auth solves

HTTP is stateless — every request is independent. The server has no memory of previous requests. Without auth, every user looks the same. Auth gives each user an identity the server can recognise and trust across multiple requests.

1.2 Session-based vs token-based auth

There are two main approaches. Understanding both helps you know why JWT is chosen for modern APIs.

	Session-based	Token-based (JWT)
Storage	Session ID stored in DB/Redis on server	No server storage — token is self-contained
Scalability	Hard to scale — all servers need same session store	Easy to scale — any server can verify the token
Logout	Delete session from DB — instant	Cannot invalidate — must wait for expiry
Mobile/API	Cookie-based — harder for mobile clients	Bearer token — works anywhere
Our choice	Not used	Used — stateless, scales well, API-friendly

1.3 Password hashing — the non-negotiable rule

Never store plain text passwords. If your database is compromised, plain text passwords expose every user's account — and since people reuse passwords, it exposes their accounts on other sites too.

Hashing is a one-way transformation. You store the hash, not the password. When a user logs in, you hash what they send and compare hashes — you never reverse the hash.

```
# plain password:      'mypassword123'
# stored in DB:        '$argon2id$v=19$m=65536,t=3,p=4$...'
```

```
# login flow:
# user sends:      'mypassword123'
# you hash it again: '$argon2id$v=19$m=65536...'
# compare hashes:  match? ✓ → login success
```

Why Argon2 over bcrypt?

Argon2 won the 2015 Password Hashing Competition and is the current recommendation. It is memory-hard (attackers need more RAM to crack), configurable, and resistant to GPU-based attacks. bcrypt is still fine but Argon2 is the modern standard. pwdlib uses Argon2 by default with `PasswordHash.recommended()`.

Timing attack prevention

If you return 'user not found' faster than 'wrong password', an attacker can determine which usernames exist just by measuring response time. The fix: always run `verify_password` even when the user doesn't exist.

```
DUMMY_HASH = password_hash.hash('dummyspassword') # computed once at startup

def authenticate_user(username, password, session):
    user = get_user(username, session)
    if not user:
        verify_password(password, DUMMY_HASH) # wastes same time
        return False                         # as real verification
    if not verify_password(password, user.hashed_password):
        return False
    return user
```

i Why this matters

Without the dummy hash check, an attacker sending 1000 requests can map out every valid username in your system in minutes — just by measuring whether the response came back in 2ms (no user) or 50ms (wrong password). This is called a timing attack.

2. JWT — JSON Web Tokens

A JWT is a compact, self-contained string that carries information (claims) and a cryptographic signature. The server issues it after login. The client sends it with every subsequent request. The server verifies the signature to trust the claims — no database lookup needed.

2.1 Structure of a JWT

A JWT has three parts separated by dots: header.payload.signature

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9          ← header (base64)
.eyJzdWIiOiJzYXVteWEiLCJleHAiOjE3MDAwMDB9      ← payload (base64)
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c ← signature (HMAC)
```

Decoded payload looks like:

```
{
  "sub": "saumya",      ← subject (username)
  "exp": 1700000000     ← expiry timestamp (Unix)
}
```

⚠ JWT is signed, not encrypted

Anyone can base64-decode the payload and read it. Never put sensitive data (passwords, credit cards) in a JWT. The signature only proves the token was issued by you — it does not hide the contents.

2.2 How the signature works

The signature is HMAC-SHA256(base64(header) + '.' + base64(payload), SECRET_KEY). If anyone tampers with the payload (e.g. changes username from 'saumya' to 'admin'), the signature no longer matches and `jwt.decode()` raises `InvalidTokenError`. This is why keeping SECRET_KEY secret is critical — whoever has it can forge any token.

2.3 Token expiry

The 'exp' claim is a Unix timestamp. PyJWT automatically checks this during decode and raises `ExpiredSignatureError` if the token is past its expiry. You set expiry at creation time:

```
def create_access_token(data: dict, expires_delta: timedelta | None = None):
```

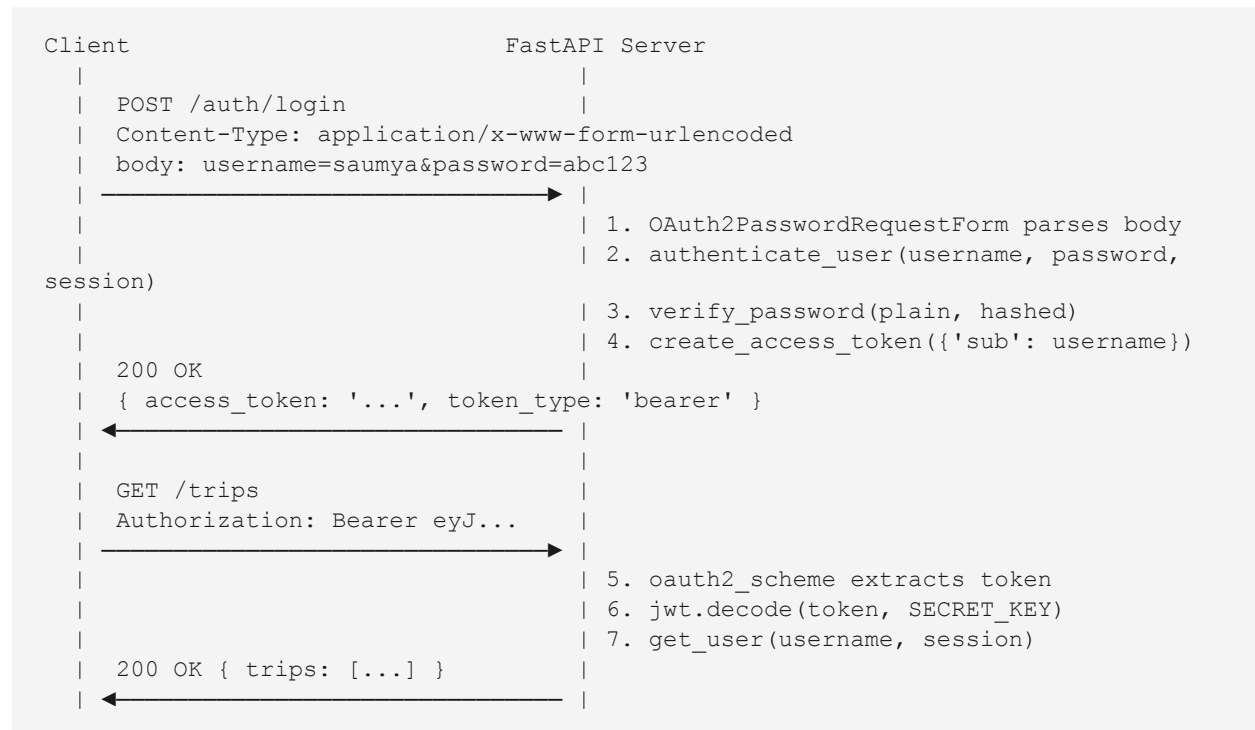
```
to_encode = data.copy()
expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=15))
to_encode.update({'exp': expire}) # add expiry to payload
return jwt.encode(to_encode, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

Typical expiry values: access tokens 15–60 minutes. Refresh tokens 7–30 days. Short access token expiry limits damage if a token is stolen.

3. OAuth2 Password Flow

OAuth2 is a specification — a set of rules for how authentication should work. FastAPI implements the OAuth2 'password flow' which is the simplest variant: username and password go directly to your server, your server returns a token.

3.1 The full login flow



3.2 OAuth2PasswordRequestForm

This is a FastAPI built-in that parses form-data (not JSON) from the request body. This is an important distinction — the login endpoint MUST receive form-data, not JSON.

```
# WRONG — sending JSON to /login
curl -X POST /auth/login -H "Content-Type: application/json" \
  -d '{"username": "saumya", "password": "abc123"}'

# CORRECT — sending form-data
curl -X POST /auth/login \
  -d "username=saumya&password=abc123"
```

In OpenAPI docs (127.0.0.1:8000/docs) the login endpoint shows a form automatically — that's the OAuth2 integration working correctly.

3.3 OAuth2PasswordBearer

This is a FastAPI dependency that extracts the Bearer token from the Authorization header automatically. You declare it once and inject it into any function that needs the raw token.

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='auth/login')

# tokenUrl tells OpenAPI docs where to send login requests
# it must match your actual login endpoint path exactly

async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)], session: SessionDep):
    payload = jwt.decode(token, settings.SECRET_KEY,
        algorithms=[settings.ALGORITHM])
    username = payload.get('sub')
    ...
```

4. How It Was Implemented — Wandr App

The traveller app (Wandr) needed auth so that each user's trips and AI memories are private. Here is the complete implementation with explanation of every decision.

4.1 File structure

```
app/
├── main.py          # app creation + include_router
├── database.py      # engine, get_session
├── models.py        # User, Trip SQLAlchemy tables
├── auth.py          # all auth logic + /register + /login routes
├── config.py        # Settings loaded from .env
└── schemas.py       # Token, TokenData (non-DB Pydantic models)
```

4.2 User models

```
class UserBase(SQLModel):
    username: str = Field(unique=True, index=True) # index speeds up lookup
    email: str = Field(unique=True, index=True)

class User(UserBase, table=True): # actual DB table
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str # NEVER plain text
    is_active: bool = Field(default=True)

class UserCreate(SQLModel): # what client sends on register
    username: str
    email: str
    password: str # plain - hashed before saving

class UserPublic(UserBase): # what server returns - no password
    id: int
```

i Why separate models?

UserCreate accepts plain password. User stores hashed password. UserPublic returns neither. If you use one model for everything, you risk accidentally returning hashed_password in API responses or allowing clients to set it directly.

4.3 Settings from .env

```
# config.py
```



```

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    SECRET_KEY: str
    ALGORITHM: str
    ACCESS_TOKEN_EXPIRE_MINUTES: int

    class Config:
        env_file = '.env'

settings = Settings() # loads automatically from .env

# .env file (never commit this):
# SECRET_KEY=your-very-long-random-secret-key-here
# ALGORITHM=HS256
# ACCESS_TOKEN_EXPIRE_MINUTES=30

```

4.4 Register endpoint

```

@router.post('/register', response_model=UserPublic)
def register(user: UserCreate, session: SessionDep):
    # check username not taken
    existing = session.execute(
        select(User).where(User.username == user.username)
    ).scalars().first()
    if existing:
        raise HTTPException(status_code=400, detail='Username already taken')

    # check email not taken
    existing_email = session.execute(
        select(User).where(User.email == user.email)
    ).scalars().first()
    if existing_email:
        raise HTTPException(status_code=400, detail='Email already taken')

    db_user = User(
        username=user.username,
        email=user.email,
        hashed_password=get_password_hash(user.password), # hash here
    )
    session.add(db_user)
    session.commit()
    session.refresh(db_user)
    return db_user # UserPublic - no password in response

```

4.5 Login endpoint

```

@router.post('/login')

```

```

def login(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
    session: SessionDep
) -> Token:
    user = authenticate_user(form_data.username, form_data.password, session)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail='Incorrect username or password',
            headers={'WWW-Authenticate': 'Bearer'},
        )
    access_token = create_access_token(
        data={'sub': user.username},
        expires_delta=timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    return Token(access_token=access_token, token_type='bearer')

```

4.6 Protecting routes with the dependency chain

FastAPI's dependency injection makes protecting routes a one-liner. The chain: token from header → decode JWT → fetch user → check active.

```

# dependency chain:
# oauth2_scheme → get_current_user → get_current_active_user

CurrentUser = Annotated[User, Depends(get_current_active_user)]

# protecting any endpoint:
@router.get('/trips')
def get_my_trips(current_user: CurrentUser, session: SessionDep):
    trips = session.execute(
        select(Trip).where(Trip.user_id == current_user.id)
    ).scalars().all()
    return trips

# getting your own profile:
@router.get('/me', response_model=UserPublic)
def get_me(current_user: CurrentUser):
    return current_user

```

4.7 Registering the router in main.py

```

# main.py
from app.auth import router as auth_router

app = FastAPI(lifespan=lifespan)
app.include_router(auth_router, prefix='/auth', tags=['auth'])

```

```
# routes become:  
# POST /auth/register  
# POST /auth/login
```

5. Problems Faced — and How to Fix Them

Every problem below was hit during actual implementation. These are the most common points where first-time auth builders get stuck.

5.1 BaseSettings import moved in Pydantic v2

⚠ Common crash on startup

ImportError: cannot import name 'BaseSettings' from 'pydantic'

In Pydantic v1, BaseSettings was in pydantic. In Pydantic v2 (which FastAPI now uses by default), it moved to a separate package.

```
# WRONG (Pydantic v1)
from pydantic import BaseSettings

# CORRECT (Pydantic v2)
from pydantic_settings import BaseSettings

# install the separate package:
pip install pydantic-settings
```

5.2 Session not passed to helper functions

⚠ Common runtime error

TypeError: get_user() missing 1 required positional argument: 'session'

FastAPI injects the session via Depends() at the endpoint level. Helper functions called from endpoints do not get the session automatically — you must pass it explicitly.

```
# WRONG — session never reaches get_user
def get_user(username: str): # no session parameter
    ...

def authenticate_user(username, password):
    user = get_user(username) # crashes

# CORRECT — pass session all the way down
def get_user(username: str, session: Session): # session here
    return session.execute(select(User).where(User.username ==
username)).scalars().first()

def authenticate_user(username, password, session: Session): # and here
```

```
user = get_user(username, session) # pass it through
```

5.3 tokenUrl mismatch breaks OpenAPI docs

⚠ Symptom

Clicking 'Authorize' in /docs sends request to wrong URL — 401 or 404 response

```
# if your login route is: POST /auth/login
# tokenUrl must be:      'auth/login' (no leading slash)

# WRONG
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='token')

# CORRECT
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='auth/login')
```

5.4 .scalars().first() vs .first()

⚠ Symptom

Comparison fails silently — user object behaves unexpectedly

```
# WRONG — returns a Row tuple, not the User object
user = session.execute(select(User).where(...)).first()
print(type(user)) # <class 'sqlalchemy.engine.row.Row'>

# CORRECT — returns the actual User instance
user = session.execute(select(User).where(...)).scalars().first()
print(type(user)) # <class 'app.models.User'>
```

5.5 Missing yield in lifespan — app never starts

⚠ Symptom

App starts but all requests hang or return 500 immediately

```
# WRONG — no yield means app never actually starts
@asynccontextmanager
```

```

async def lifespan(app: FastAPI):
    create_db_and_tables()
    # missing yield!

# CORRECT
@asynccontextmanager
async def lifespan(app: FastAPI):
    create_db_and_tables() # startup
    yield                 # app runs here
    # shutdown code goes here if needed

```

5.6 Test engine not being used by endpoints

⚠ Symptom

Tests fail with 'no such table' even though create_all was called

The root cause: `get_session` closes over the engine variable at definition time. Replacing the engine object later doesn't affect already-defined functions. The fix is to make `get_session` look up the engine dynamically at call time.

```

# database.py - dynamic lookup fix
def get_session():
    import app.database as db_module      # import the module
    with Session(db_module.engine) as session: # read engine at call time
        yield session

# conftest.py - swap engine before any imports
import app.database as db_module
test_engine = create_engine('sqlite:///memory:', ...)
db_module.engine = test_engine # must happen before app imports

from app.main import app # now app uses test_engine

```

6. Advanced Topics

The implementation above is production-ready for a small app. As the app grows, these are the next problems you'll face and how to solve them.

6.1 Refresh tokens

Access tokens expire (30 min). Without refresh tokens, users must re-login every 30 minutes. Refresh tokens are long-lived tokens (7–30 days) that can request a new access token without re-entering credentials.

```
# flow with refresh tokens:
# 1. Login → server returns access_token (30min) + refresh_token (7 days)
# 2. Access token expires → client sends refresh_token to POST /auth/refresh
# 3. Server validates refresh_token → issues new access_token
# 4. Refresh token expires → user must login again

# store refresh tokens in DB so you can invalidate them on logout
class RefreshToken(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    token: str = Field(unique=True, index=True)
    user_id: int = Field(foreign_key='user.id')
    expires_at: datetime
    is_revoked: bool = Field(default=False)
```

6.2 Logout and token blacklisting

JWT is stateless — you cannot invalidate a token before it expires. If a user logs out or their token is stolen, the token still works until expiry. Solutions:

- Short access token expiry (15 min) — limits damage window
- Token blacklist in Redis — store revoked tokens, check on every request
- Refresh token revocation — delete from DB on logout

```
# Redis blacklist approach
import redis
r = redis.Redis()

@router.post('/logout')
def logout(token: Annotated[str, Depends(oauth2_scheme)]):
    payload = jwt.decode(token, settings.SECRET_KEY,
        algorithms=[settings.ALGORITHM])
    exp = payload.get('exp')
    ttl = exp - int(datetime.now(timezone.utc).timestamp())
```

```

    r.setex(f'blacklist:{token}', ttl, 'revoked') # expires with token
    return {'message': 'logged out'}

# check in get_current_user:
if r.exists(f'blacklist:{token}'):
    raise credentials_exception

```

6.3 Rate limiting on login

Without rate limiting, attackers can try millions of password combinations. Add `slowapi` to limit login attempts per IP.

```

pip install slowapi

from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@router.post('/login')
@limiter.limit('5/minute') # max 5 login attempts per minute per IP
def login(request: Request, form_data: ...):
    ...

```

6.4 Role-based access control (RBAC)

As the app grows, you'll need different permission levels — admin can see all users' trips, regular user can only see their own.

```

# add role to User model
class UserRole(str, Enum):
    user = 'user'
    admin = 'admin'

class User(UserBase, table=True):
    ...
    role: UserRole = Field(default=UserRole.user)

# admin-only dependency
def require_admin(current_user: CurrentUser):
    if current_user.role != UserRole.admin:
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

```



```
AdminUser = Annotated[User, Depends(require_admin)]

@router.get('/admin/users')
def get_all_users(admin: AdminUser, session: SessionDep):
    return session.execute(select(User)).scalars().all()
```

6.5 OAuth2 with third-party providers (Google, GitHub)

Instead of managing passwords yourself, delegate to Google or GitHub. Users click 'Login with Google', Google verifies their identity, sends you a code, you exchange it for user info and issue your own JWT. Use Authlib for this.

```
pip install authlib httpx

# .env additions:
# GOOGLE_CLIENT_ID=...
# GOOGLE_CLIENT_SECRET=...

# flow:
# GET /auth/google → redirect to Google login page
# GET /auth/google/callback → Google sends code → exchange for user info → issue
# JWT
```

6.6 Testing authenticated endpoints

Protected endpoints need a valid token in the test request. Create a helper that generates a test token and inject it in headers.

```
# conftest.py
from app.auth import create_access_token
from datetime import timedelta

@pytest.fixture()
def auth_client(client):
    # create a test user
    client.post('/auth/register', json={
        'username': 'testuser',
        'email': 'test@test.com',
        'password': 'testpass123'
    })
    # generate token directly (no login request needed)
    token = create_access_token({'sub': 'testuser'}, timedelta(minutes=30))
    client.headers['Authorization'] = f'Bearer {token}'
    return client
```

```
# using it:
def test_get_my_trips(auth_client):
    response = auth_client.get('/trips')
    assert response.status_code == 200
```

6.7 CORS — Cross Origin Resource Sharing

When you add a Next.js frontend (week 7), the browser will block API requests from localhost:3000 to localhost:8000 by default. CORS headers tell the browser which origins are allowed.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000', 'https://your-app.vercel.app'],
    allow_credentials=True, # allows cookies and auth headers
    allow_methods=['*'],
    allow_headers=['*'],
)
```

6.8 Storing tokens securely on the client

Where you store the JWT on the frontend matters for security:

Storage	Risk	Use when
localStorage	XSS attacks can steal token	Simple SPAs, low-security apps
sessionStorage	XSS — cleared on tab close	Temporary sessions
HttpOnly Cookie	CSRF attacks (mitigated with SameSite)	Production apps — recommended
Memory (variable)	Lost on page refresh	Highest security, combined with refresh token in cookie

7. Quick Reference

Key dependencies

pwdlib[argon2]	Password hashing with Argon2
PyJWT	JWT encoding and decoding (import jwt)
pydantic-settings	BaseSettings — loads .env automatically
python-multipart	Required for OAuth2PasswordRequestForm to work
slowapi	Rate limiting middleware
authlib	Third-party OAuth (Google, GitHub)

Status codes to use

400 Bad Request	Username already taken, email already exists
401 Unauthorized	Invalid credentials, invalid/expired token
403 Forbidden	Valid token but insufficient permissions (admin only)
422 Unprocessable	Validation error — wrong field types or missing required fields

Environment variables

```
# .env — never commit to git
SECRET_KEY=generate-with: openssl rand -hex 32
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# .env.example — commit this
SECRET_KEY=
ALGORITHM=
ACCESS_TOKEN_EXPIRE_MINUTES=
```

Generate a secure SECRET_KEY

```
# in terminal:
openssl rand -hex 32
# output: a8f3c2e1d4b7f9a2c5e8d1b4f7a0c3e6d9b2f5a8c1e4d7b0f3a6c9e2d5b8f1a4
```

Built while building Wandr — week 3 of the remote job roadmap